

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Member: 허다운, 성지석,
최재규

Presenter: 최재규

Contents

1.Introduction

2.Background

3.Preliminary

4.BERT: Architecture & Training method

5.Experiments

6.Conclusion

Introduction

Definition

BERT: Bidirectional Encoder Representations from Transformer

BERT is a method of pre-training language representations, meaning that we train a general-purpose “language understanding” model on a large text corpus (BooksCorpus and Wikipedia), and then use that model for downstream NLP tasks (fine tuning) that we care about (like question answering—SQuAD).

BERT performance

2019.07.26기준

SQuAD1.1 Leaderboard

Here are the ExactMatch (EM) and F1 scores evaluated on the test set of SQuAD v1.1.

Rank	Model	EM	F1
	Human Performance <i>Stanford University</i> <i>(Rajpurkar et al. '16)</i>	82.304	91.221
1 May 21, 2019	XLNet (single model) <i>Google Brain & CMU</i>	89.898	95.080
2 Jul 21, 2019	SpanBERT (single model) <i>FAIR & UW</i>	88.839	94.635
3 Jul 03, 2019	BERT+WWM+MT (single model) <i>Xiao Research</i>	88.650	94.393
4 Jul 21, 2019	Tuned BERT-1seq Large Cased (single model) <i>FAIR & UW</i>	87.465	93.294
5 Oct 05, 2018	BERT (ensemble) <i>Google AI Language</i> https://arxiv.org/abs/1810.04805	87.433	93.160
6 May 14, 2019	ATB (single model) <i>Anonymous</i>	86.940	92.641
7 Jul 21, 2019	Tuned BERT Large Cased (single model) <i>FAIR & UW</i>	86.521	92.617
7 Jul 04, 2019	BERT+MT (single model) <i>Xiao Research</i>	86.458	92.645
8 Feb 14, 2019	KT-NET (single model) <i>Baidu NLP</i>	85.944	92.425
8 Sep 26, 2018	nlnet (ensemble) <i>Microsoft Research Asia</i>	85.954	91.677
8 Feb 28, 2019	ST-bl <i>single model</i>	85.430	91.976
9 Mar 14, 2019	BISAN (single model) <i>Seoul National University & Hyundai Motors</i>	85.314	91.756

Background

Transfer learning

What is Transfer learning?

- **Transfer learning** is a research problem in [machine learning](#) that focuses on storing knowledge gained while solving one problem and applying it to a different but related problem.

Transfer learning

Classification of transfer learning

- **Feature based approach**

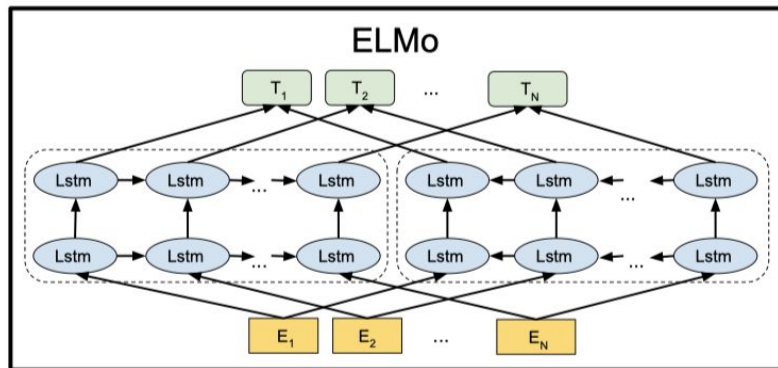
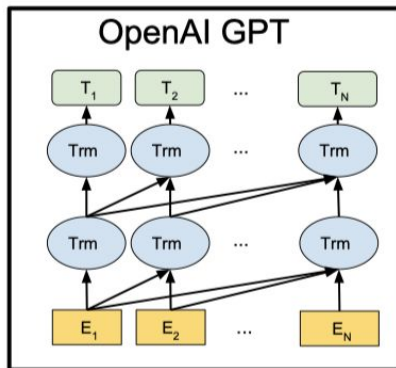
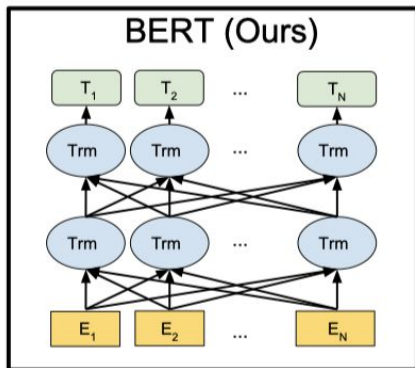
Training feature vectors in advance for words

- **Fine-tuning approach**

Training in advance how to extract features

Differences

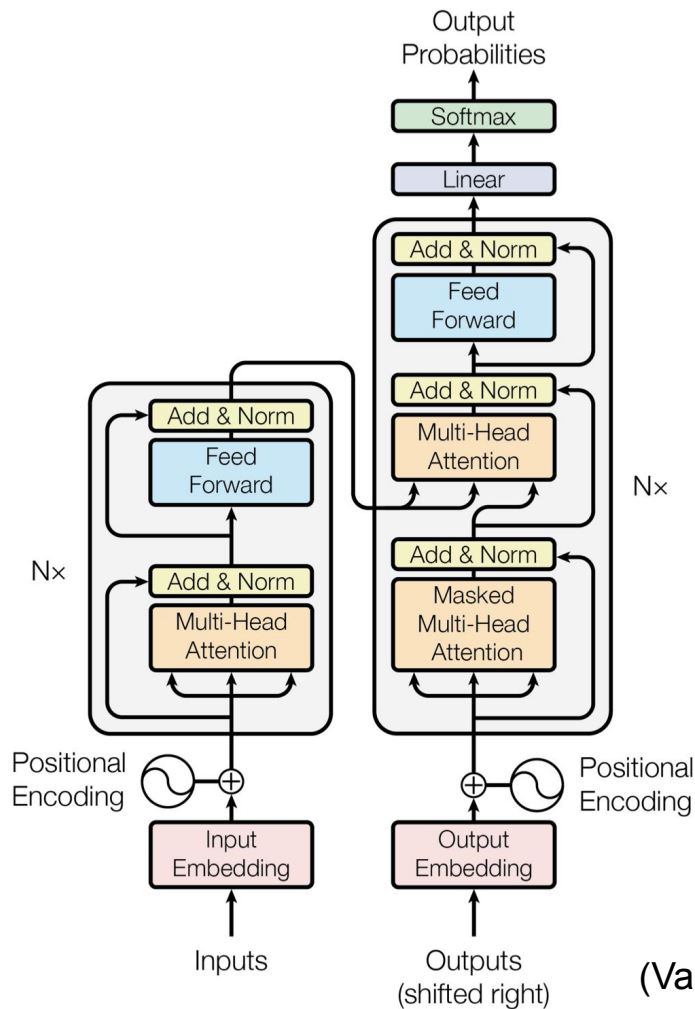
- Bert uses a bidirectional Transformer
- OpenAI GPT Uses a left-to-right Transformer
- ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTMs to generate features for downstream tasks



Preliminary

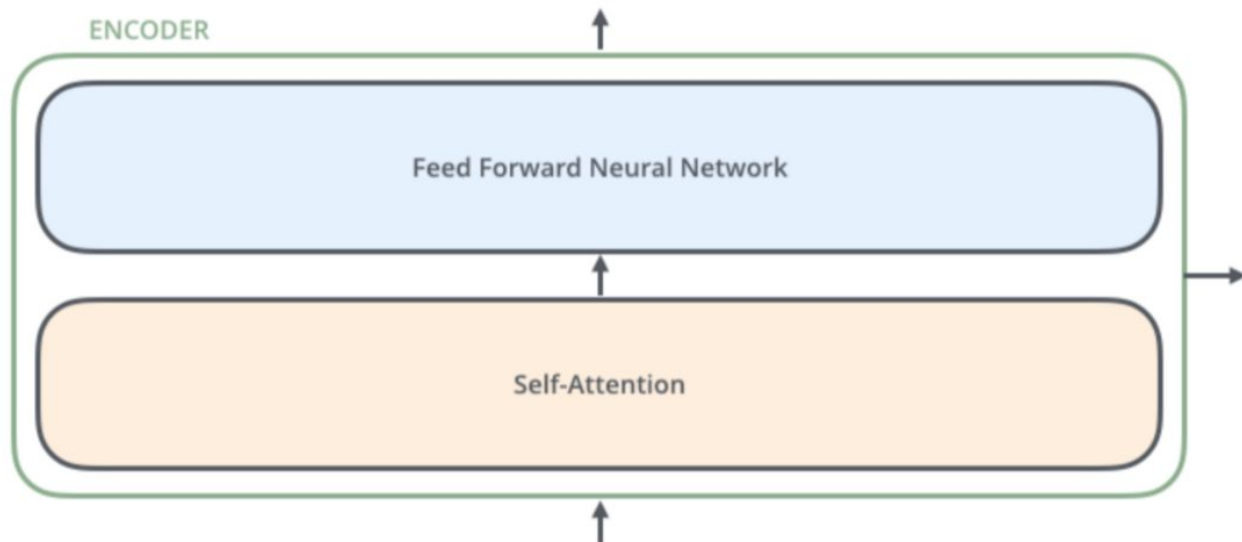
Transformer

Model architecture

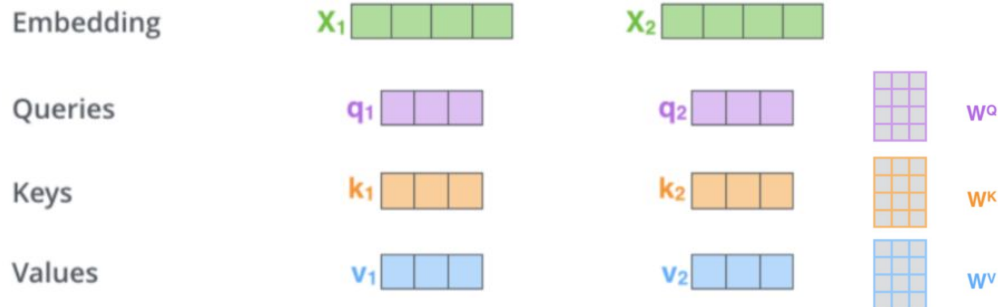
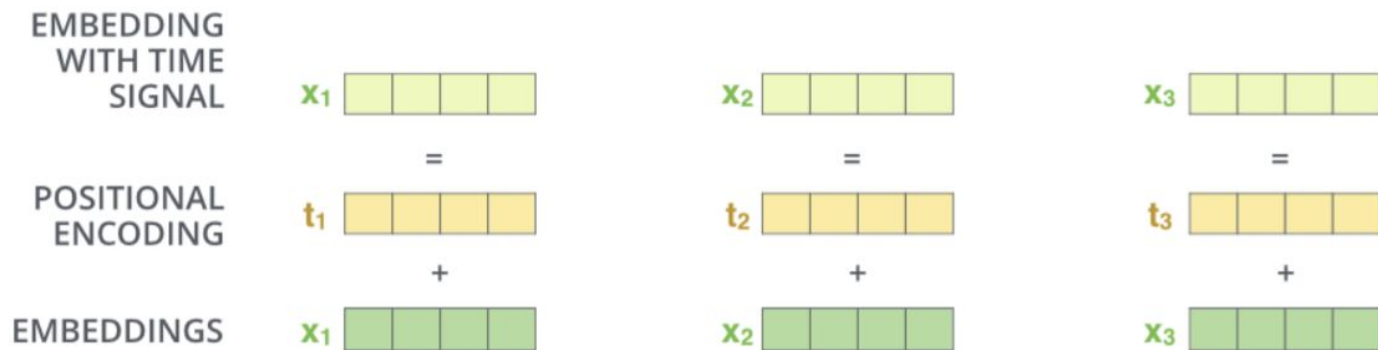


(Vaswani et al. 2017)

Encoder

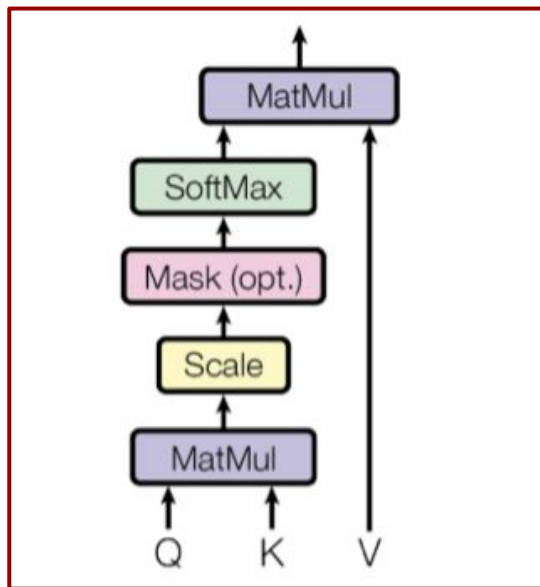


Self-Attention - Input

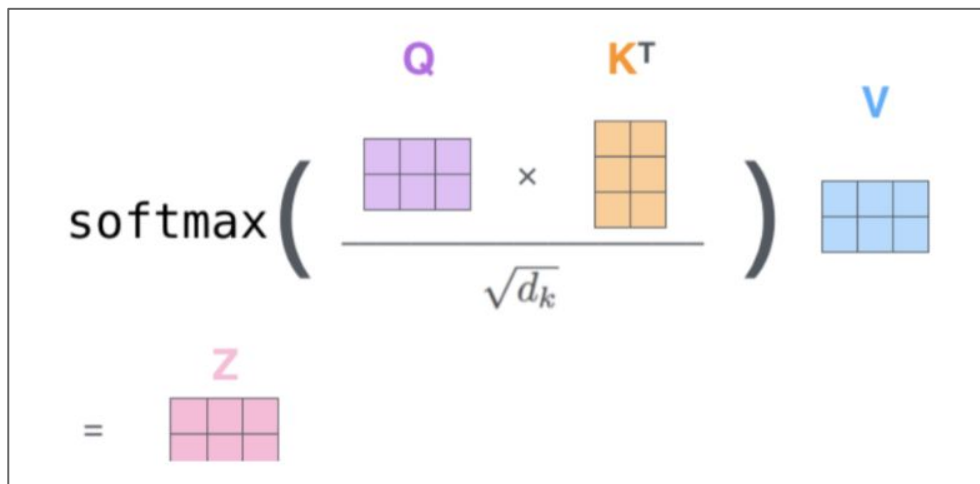


Self-Attention

Scaled Dot-Product
Attention

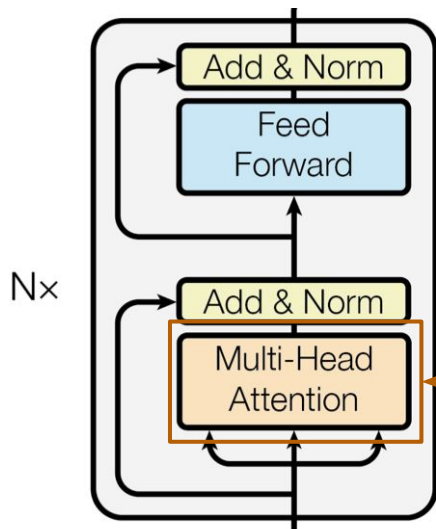


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

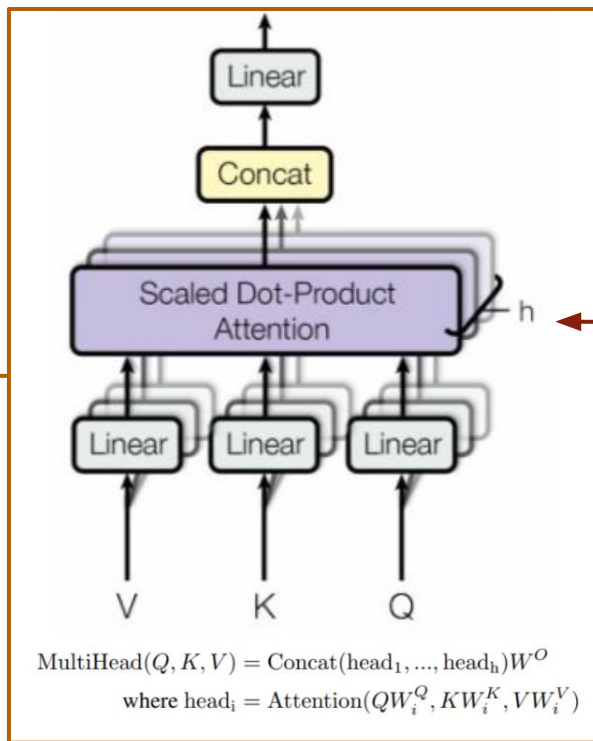


Self-Attention

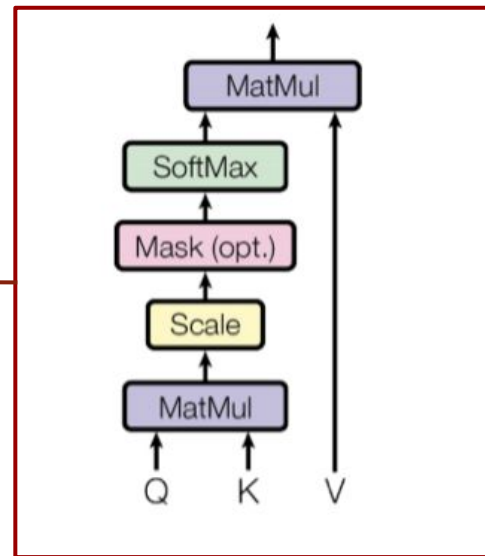
Transformer - Encoder



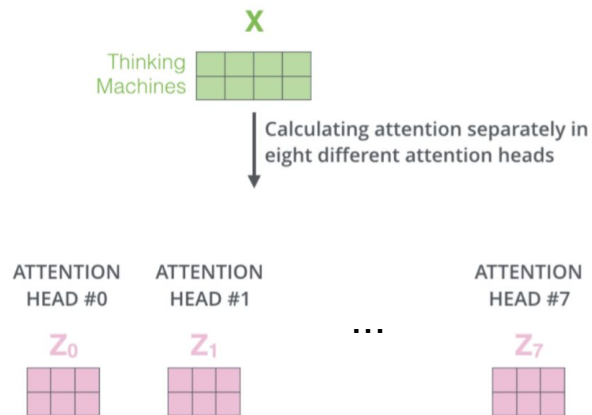
Multi-Head Attention



Scaled Dot-Product Attention



Multi-Head Attention



1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

$\times$

3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



Multi-Head Attention

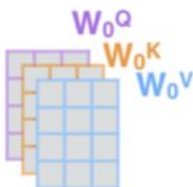
1) This is our input sentence*

Thinking
Machines

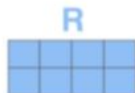
2) We embed each word*



3) Split into 8 heads.
We multiply X or R with weight matrices



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

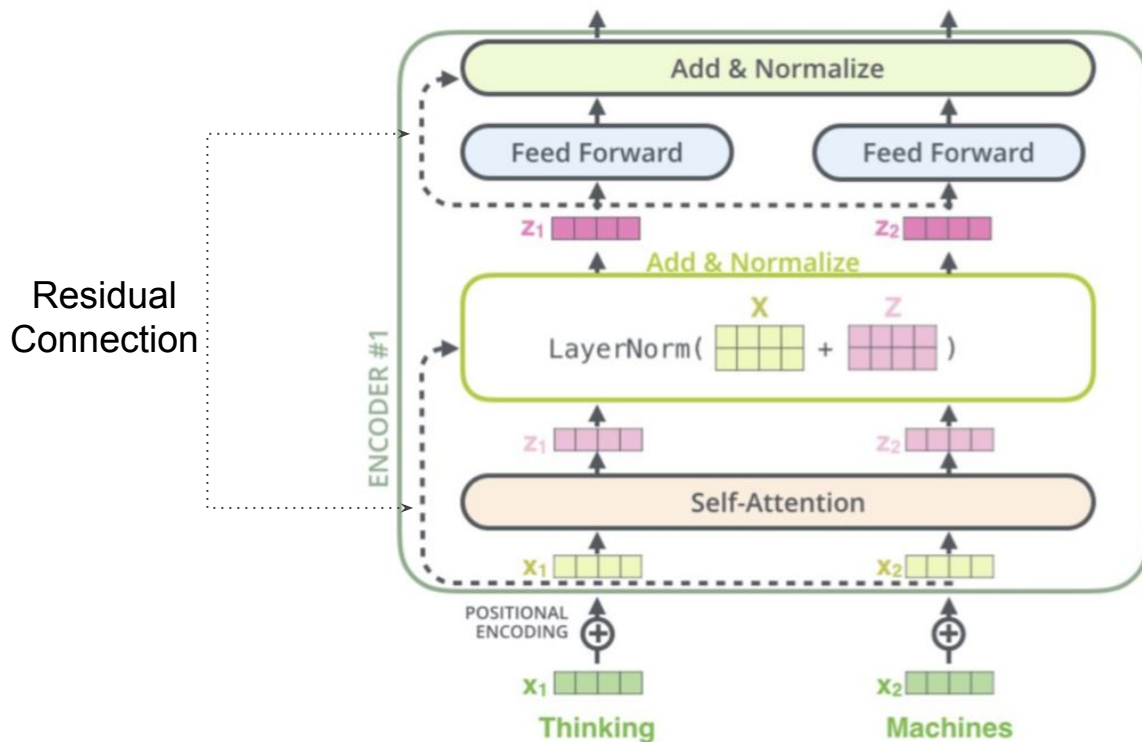


4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

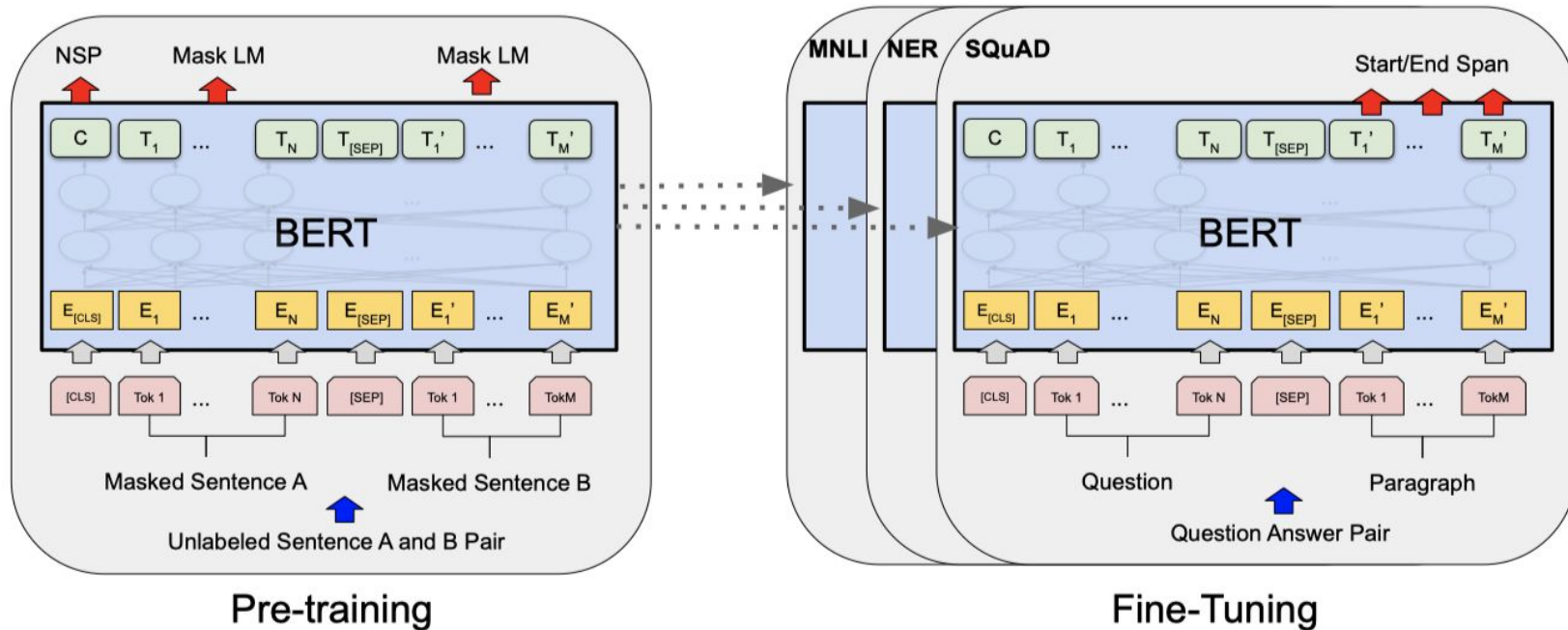


Residual Connection & Normalization

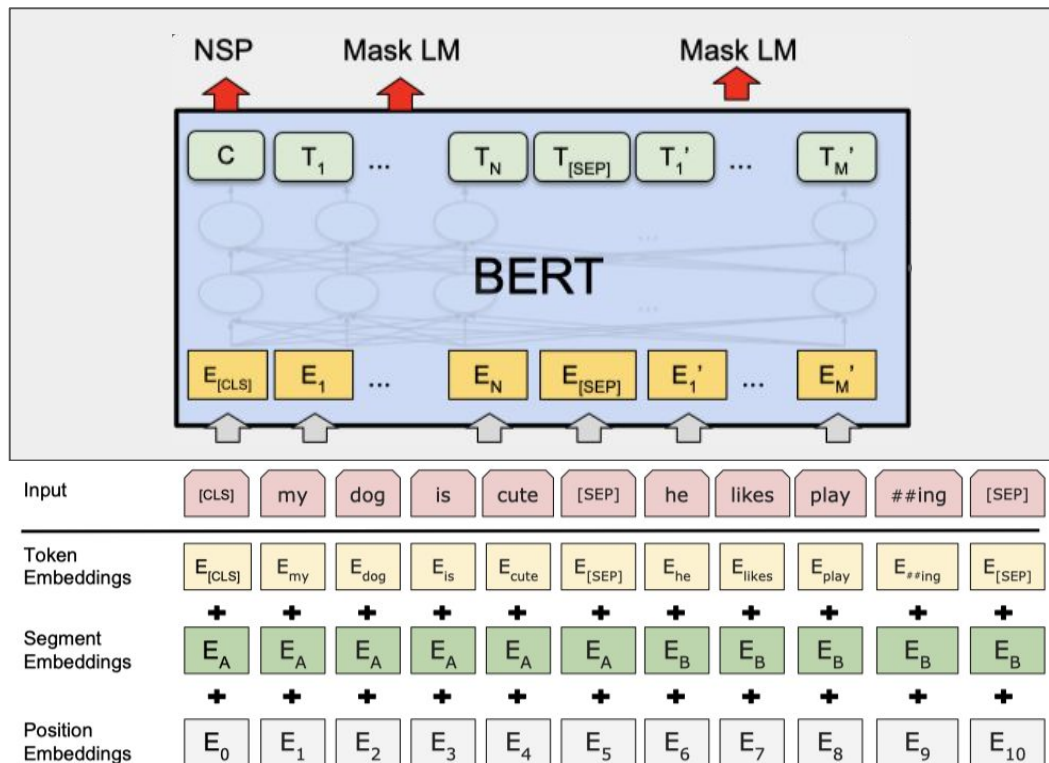


BERT architecture & Training method

Pre-training & fine-tuning procedures for BERT

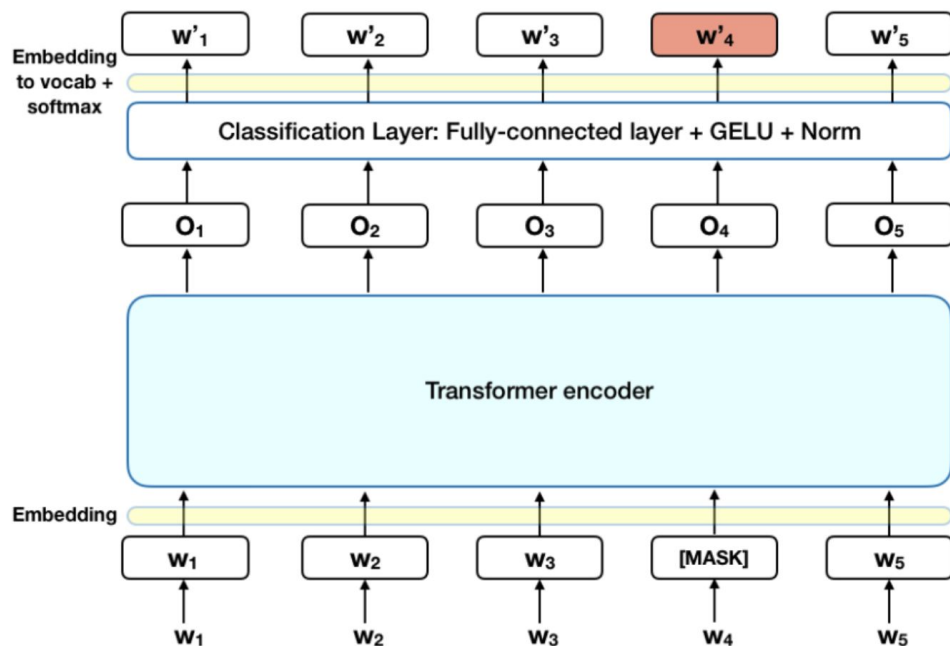


Input/Output Representations



Pre-training BERT

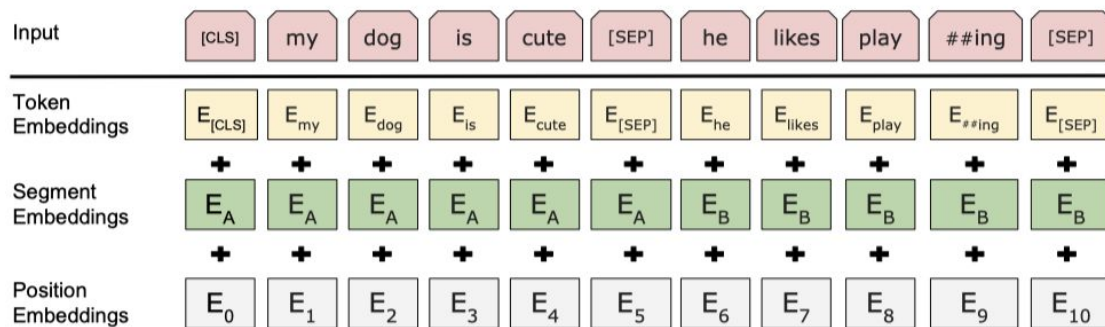
Task #1: Masked LM



- 80% of the time: Replace the word with the [MASK] token, e.g., my dog is hairy → my dog is [MASK]
- 10% of the time: Replace the word with a random word, e.g., my dog is hairy → my dog is apple
- 10% of the time: Keep the word unchanged, e.g., my dog is hairy → my dog is hairy. The purpose of this is to bias the representation towards the actual observed word.

Pre-training BERT

Task #2: Next Sentence Prediction (NSP)



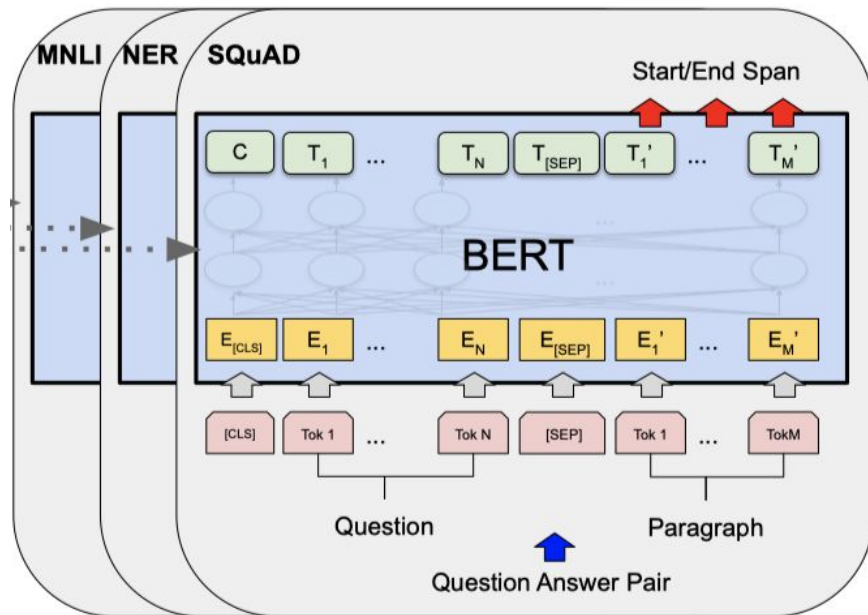
Input = [CLS] the man went to [MASK] store [SEP]
he bought a gallon [MASK] milk [SEP]

Label = IsNext

Input = [CLS] the man [MASK] to the store [SEP]
penguin [MASK] are flight ##less birds [SEP]

Label = NotNext

Fine-tuning BERT



Fine-Tuning

- **Batch size:** 16, 32
- **Learning rate (Adam):** $5e-5$, $3e-5$, $2e-5$
- **Number of epochs:** 2, 3, 4

Experiments

Experiments

General Language Understanding Evaluation (GLUE)

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

- 모든 경우에서 BERT(LARGE)가 SOTA 달성
- OpenAI GPT와 BERT(LARGE)는 attention masking을 제외하고 model architecture가 유사하지만 BERT(LARGE)가 좋은 성능을 보여줌
- BERT(BASE)보다 BERT(LARGE)가 좋은 성능을 보여줌

Experiments

SQuAD, SWAG

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	-	-	82.3	91.2
#1 Ensemble - nlnet	-	-	86.0	91.7
#2 Ensemble - QANet	-	-	84.5	90.5
Published				
BiDAF+ELMo (Single)	-	85.6	-	85.8
R.M. Reader (Ensemble)	81.2	87.9	82.3	88.5
Ours				
BERT _{BASE} (Single)	80.8	88.5	-	-
BERT _{LARGE} (Single)	84.1	90.9	-	-
BERT _{LARGE} (Ensemble)	85.8	91.8	-	-
BERT _{LARGE} (Sgl.+TriviaQA)	84.2	91.1	85.1	91.8
BERT _{LARGE} (Ens.+TriviaQA)	86.2	92.2	87.4	93.2

Table 2: SQuAD 1.1 results. The BERT ensemble is 7x systems which use different pre-training checkpoints and fine-tuning seeds.

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	86.3	89.0	86.9	89.5
#1 Single - MIR-MRC (F-Net)	-	-	74.8	78.0
#2 Single - nlnet	-	-	74.2	77.1
Published				
unet (Ensemble)	-	-	71.4	74.9
SLQA+ (Single)	-	-	71.4	74.4
Ours				
BERT _{LARGE} (Single)	78.7	81.9	80.0	83.1

Table 3: SQuAD 2.0 results. We exclude entries that use BERT as one of their components.

System	Dev	Test
ESIM+GloVe	51.9	52.7
ESIM+ELMo	59.1	59.2
OpenAI GPT	-	78.0
BERT _{BASE}	81.6	-
BERT _{LARGE}	86.6	86.3
Human (expert) [†]	-	85.0
Human (5 annotations) [†]	-	88.0

Table 4: SWAG Dev and Test accuracies. [†]Human performance is measured with 100 samples, as reported in the SWAG paper.

- SOTA 달성
- SQuAD 1.1, SWAG의 경우 사람보다 좋은 결과를 보여줌

Ablation Studies

Ablation Studies

Effect of Pre-training Tasks

Tasks	MNLI-m (Acc)	Dev Set			
		QNLI (Acc)	MRPC (Acc)	SST-2 (Acc)	SQuAD (F1)
BERT _{BASE}	84.4	88.4	86.7	92.7	88.5
No NSP	83.9	84.9	86.5	92.6	87.9
LTR & No NSP	82.1	84.3	77.5	92.1	77.8
+ BiLSTM	82.1	84.1	75.7	91.6	84.9

Table 5: Ablation over the pre-training tasks using the BERT_{BASE} architecture. “No NSP” is trained without the next sentence prediction task. “LTR & No NSP” is trained as a left-to-right LM without the next sentence prediction, like OpenAI GPT. “+ BiLSTM” adds a randomly initialized BiLSTM on top of the “LTR + No NSP” model during fine-tuning.

Effect of Model Size

	Hyperparams				Dev Set Accuracy		
	#L	#H	#A	LM (ppl)	MNLI-m	MRPC	SST-2
Base	3	768	12	5.84	77.9	79.8	88.4
	6	768	3	5.24	80.6	82.2	90.7
	6	768	12	4.68	81.9	84.8	91.3
	12	768	12	3.99	84.4	86.7	92.9
	12	1024	16	3.54	85.7	86.9	93.3
Large	24	1024	16	3.23	86.6	87.8	93.7

Table 6: Ablation over BERT model size. #L = the number of layers; #H = hidden size; #A = number of attention heads. “LM (ppl)” is the masked LM perplexity of held-out training data.

Ablation Studies

Feature-based Approach with BERT

System	Dev F1	Test F1
ELMo (Peters et al., 2018a)	95.7	92.2
CVT (Clark et al., 2018)	-	92.6
CSE (Akbik et al., 2018)	-	93.1
Fine-tuning approach		
BERT _{LARGE}	96.6	92.8
BERT _{BASE}	96.4	92.4
Feature-based approach (BERT _{BASE})		
Embeddings	91.0	-
Second-to-Last Hidden	95.6	-
Last Hidden	94.9	-
Weighted Sum Last Four Hidden	95.9	-
Concat Last Four Hidden	96.1	-
Weighted Sum All 12 Layers	95.5	-

Table 7: CoNLL-2003 Named Entity Recognition results. Hyperparameters were selected using the Dev set. The reported Dev and Test scores are averaged over 5 random restarts using those hyperparameters.

Conclusion

State-of-the-art (SOTA)

Bidirectional model

Performance of model BERT_{LARGE} is better than model BERT_{BASE}

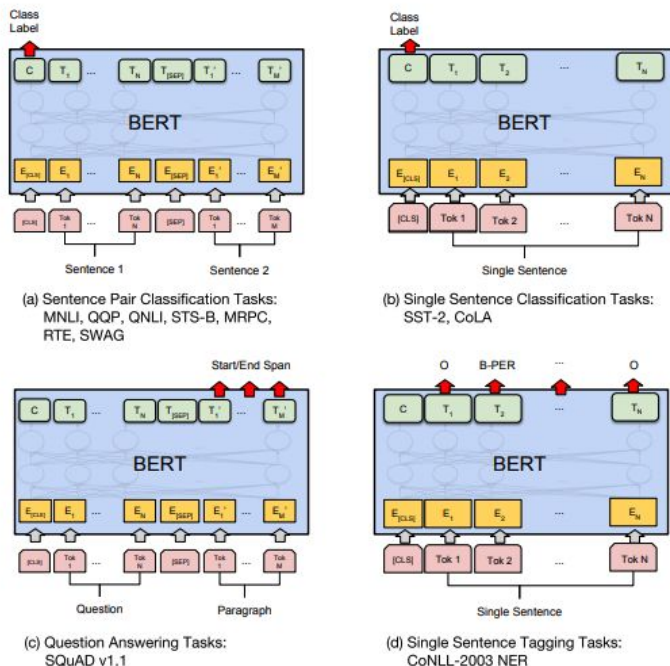
Reference

- <https://mino-park7.github.io/nlp/2018/12/12/bert-논문정리/?fbclid=IwAR3S-8iLWEVG6FGUVxoYdwQyA-zG0GpOUzVEsFBd0ARFg4eFXqCyGLznu7w#35-fine-tuning-procedure>
- <http://jalammar.github.io/illustrated-transformer/>
- <https://blog.naver.com/bcj1210/221602723234>
- <https://medium.com/dissecting-bert>
- 김기현의 자연어처리 딥러닝 캠프
- Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems*. 2017.

Q & A

Experiments

Overview



- **Classification**
 - input sequence에 대해서 일정한 차원수의 representation 결과를 얻고 싶기 때문에, [CLS] token의 **Transformer output**값을 사용
 - [CLS] token의 벡터는 H차원을 가짐.
 - 여기서 **classify**하고 싶은 갯수(K)에 따라 **classification layer**를 추가
 - label probabilities는 **standard softmax**로 계산
- **Q&A task**
 - QA task경우는, **Question**에 정답이 되는 **Paragraph**의 **substring**을 뽑아내는 것이므로, [SEP] token 이후의 token들에서 **Start/End Span**을 찾아내는 task를 수행
- **NER**
 - single sentence에서 각 토큰이 어떤 class를 갖는지 모두 **classifier** 적용

Activation function

Gaussian Error Linear Units (GELU)

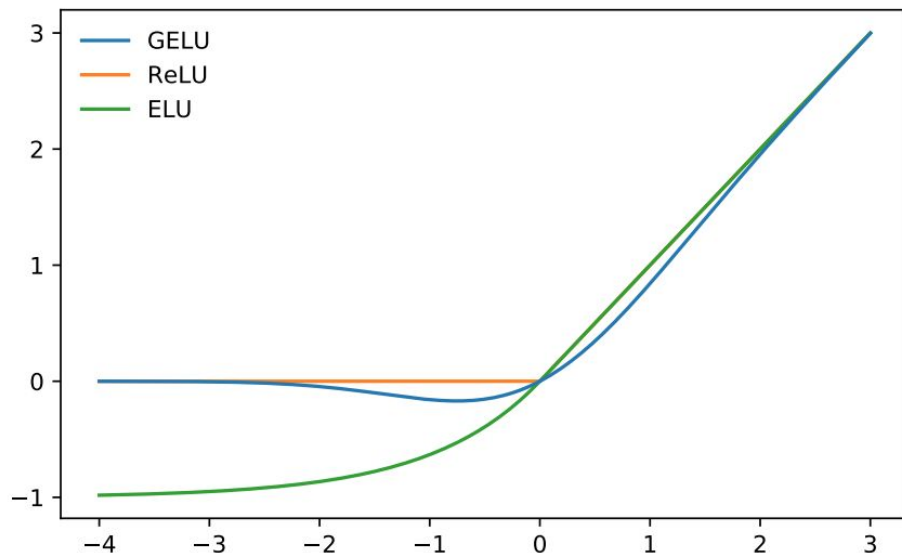


Figure 1: The GELU ($\mu = 0, \sigma = 1$), ReLU, and ELU ($\alpha = 1$).